

ANSI SQL Using MySQL

1. User Upcoming Events

```
SELECT e.event_id, e.title, e.city, e.start_date
FROM Events e
JOIN Registrations r ON r.event_id = e.event_id
JOIN Users u ON u.user_id = r.user_id
WHERE e.status = 'upcoming'
      AND e.city = u.city
      AND u.user_id = 1
ORDER BY e.start_date;
```

2. Top Rated Events

```
SELECT e.event_id, e.title, AVG(f.rating) AS avg_rating, COUNT(f.feedback_id) AS
feedback_count
FROM Events e
JOIN Feedback f ON f.event_id = e.event_id
GROUP BY e.event_id, e.title
HAVING COUNT(f.feedback_id) >= 10
ORDER BY avg_rating DESC;
```

3. Inactive Users

```
SELECT u.user_id, u.full_name, u.email, u.city
FROM Users u
WHERE u.user_id NOT IN (
      SELECT r.user_id
```

```
FROM Registrations r

WHERE r.registration_date >= DATE_SUB(CURDATE(), INTERVAL 90 DAY)

);
```

4. Peak Session Hours

```
SELECT event_id, COUNT(*) AS session_count

FROM Sessions

WHERE TIME(start_time) >= '10:00:00'

AND TIME(start_time) < '12:00:00'

GROUP BY event_id;
```

5. Most Active Cities

```
SELECT u.city, COUNT(DISTINCT r.user_id) AS distinct_users

FROM Registrations r

JOIN Users u ON u.user_id = r.user_id

GROUP BY u.city

ORDER BY distinct_users DESC

LIMIT 5;
```

6. Event Resource Summary

```
SELECT e.event_id, e.title,

       SUM(res.resource_type = 'pdf') AS pdf_count,

       SUM(res.resource_type = 'image') AS image_count,

       SUM(res.resource_type = 'link') AS link_count,

       COUNT(res.resource_id) AS total_resources

FROM Events e

LEFT JOIN Resources res ON res.event_id = e.event_id
```

```
GROUP BY e.event_id, e.title;
```

7. Low Feedback Alerts

```
SELECT u.user_id, u.full_name, e.title AS event_name, f.rating, f.comments  
  
FROM Feedback f  
  
JOIN Users u ON u.user_id = f.user_id  
  
JOIN Events e ON e.event_id = f.event_id  
  
WHERE f.rating < 3;
```

8. Sessions per Upcoming Event

```
SELECT e.event_id, e.title, COUNT(s.session_id) AS session_count  
  
FROM Events e  
  
LEFT JOIN Sessions s ON s.event_id = e.event_id  
  
WHERE e.status = 'upcoming'  
  
GROUP BY e.event_id, e.title;
```

9. Organizer Event Summary

```
SELECT u.user_id, u.full_name, e.status, COUNT(*) AS event_count  
  
FROM Events e  
  
JOIN Users u ON u.user_id = e.organizer_id  
  
GROUP BY u.user_id, u.full_name, e.status  
  
ORDER BY u.full_name;
```

10. Feedback Gap

```
SELECT DISTINCT e.event_id, e.title  
  
FROM Events e  
  
JOIN Registrations r ON r.event_id = e.event_id  
  
WHERE e.event_id NOT IN (  
  
    SELECT event_id FROM Feedback  
  
);
```

11. Daily New User Count

```
SELECT registration_date, COUNT(*) AS new_users  
  
FROM Users  
  
WHERE registration_date >= DATE_SUB(CURDATE(), INTERVAL 7 DAY)  
  
GROUP BY registration_date  
  
ORDER BY registration_date;
```

12. Event with Maximum Sessions

```
SELECT event_id, COUNT(*) AS session_count  
  
FROM Sessions  
  
GROUP BY event_id  
  
HAVING COUNT(*) = (  
  
    SELECT MAX(cnt)  
  
    FROM (SELECT COUNT(*) AS cnt FROM Sessions GROUP BY event_id) AS t  
  
);
```

13. Average Rating per City

```
SELECT e.city, AVG(f.rating) AS avg_rating  
  
FROM Events e  
  
JOIN Feedback f ON f.event_id = e.event_id  
  
GROUP BY e.city  
  
ORDER BY avg_rating DESC;
```

14. Event Session Time Conflict

```
SELECT e.event_id, e.title, COUNT(r.registration_id) AS registration_count  
  
FROM Events e  
  
JOIN Registrations r ON r.event_id = e.event_id
```

```
GROUP BY e.event_id, e.title  
  
ORDER BY registration_count DESC  
  
LIMIT 3;
```

15. Event Session Time Conflict

```
SELECT s1.event_id,  
  
       s1.session_id AS session_1, s1.title AS session_1_title,  
  
       s2.session_id AS session_2, s2.title AS session_2_title  
  
FROM Sessions s1  
  
JOIN Sessions s2  
  
  ON s1.event_id = s2.event_id  
  
 AND s1.session_id < s2.session_id  
  
WHERE s1.start_time < s2.end_time  
  
      AND s2.start_time < s1.end_time;
```

16. Unregistered Active Users

```
SELECT u.user_id, u.full_name, u.email, u.registration_date  
  
FROM Users u  
  
WHERE u.registration_date >= DATE_SUB(CURDATE(), INTERVAL 30 DAY)  
  
      AND u.user_id NOT IN (SELECT user_id FROM Registrations);
```

17. Multi-Session Speakers

```
SELECT speaker_name, COUNT(*) AS session_count  
  
FROM Sessions  
  
GROUP BY speaker_name  
  
HAVING COUNT(*) > 1;
```

18. Resource Availability Check

```
SELECT e.event_id, e.title  
  
FROM Events e
```

```
WHERE e.event_id NOT IN (  
    SELECT event_id FROM Resources  
);
```

19. Completed Events with Feedback Summary

```
SELECT e.event_id, e.title,  
    COUNT(DISTINCT r.registration_id) AS total_registrations,  
    AVG(f.rating) AS avg_rating  
FROM Events e  
LEFT JOIN Registrations r ON r.event_id = e.event_id  
LEFT JOIN Feedback f ON f.event_id = e.event_id  
WHERE e.status = 'completed'  
GROUP BY e.event_id, e.title;
```

20. User Engagement Index

```
SELECT u.user_id, u.full_name,  
    COUNT(DISTINCT r.event_id) AS events_attended,  
    COUNT(DISTINCT f.feedback_id) AS feedback_submitted  
FROM Users u  
LEFT JOIN Registrations r ON r.user_id = u.user_id  
LEFT JOIN Feedback f ON f.user_id = u.user_id  
GROUP BY u.user_id, u.full_name;
```

21. Top Feedback Providers

```
SELECT u.user_id, u.full_name, COUNT(f.feedback_id) AS feedback_count  
FROM Users u  
JOIN Feedback f ON f.user_id = u.user_id  
GROUP BY u.user_id, u.full_name
```

ORDER BY feedback_count DESC

LIMIT 5;

22. Duplicate Registrations Check

SELECT user_id, event_id, COUNT(*) AS registration_count

FROM Registrations

GROUP BY user_id, event_id

HAVING COUNT(*) > 1;

23. Registration Trends

SELECT DATE_FORMAT(registration_date, '%Y-%m') AS registration_month,

COUNT(*) AS total_registrations

FROM Registrations

WHERE registration_date >= DATE_SUB(CURDATE(), INTERVAL 12 MONTH)

GROUP BY registration_month

ORDER BY registration_month;

24. Average Session Duration per Event

SELECT event_id, AVG(TIMESTAMPDIFF(MINUTE, start_time, end_time)) AS
avg_duration_minutes

FROM Sessions

GROUP BY event_id;

25. Events Without Sessions

SELECT e.event_id, e.title

FROM Events e

WHERE e.event_id NOT IN (

SELECT event_id FROM Sessions

);